Topological Sort and Course Schedule

1. Topological Sort

Definition: Topological sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge u → v, vertex u comes before vertex v.

Example: For edges A → B, B → C, and C → D, one valid topological order is A, B, C, D.

Methods

1. **Kahn's Algorithm (BFS):** Select vertices with indegree 0, remove them, and update the indegrees of their neighbours.
2. **DFS Method:** Visit neighbours first, then push each vertex onto a stack after its neighbours are processed. Reverse the finishing order to obtain a topological ordering.

Steps of Kahn's Algorithm

1. Calculate the indegree of every vertex.
2. Add all vertices with indegree 0 to a queue.
3. Remove one vertex from the queue and add it to the result.
4. Reduce the indegree of its adjacent vertices by 1.
5. Add any vertex whose indegree becomes 0 to the queue.
6. If all vertices are processed, the result is a topological ordering; otherwise, the graph contains a cycle.

2. Course Schedule Problem

Definition: The Course Schedule problem asks whether all courses can be completed when some courses have prerequisites.

Example: If Course 0 must be completed before Course 1, the edge is 0 → 1. A valid order is 0, 1, so the answer is true.

Approach

1. Represent courses as vertices in a directed graph.
2. Draw an edge from each prerequisite course to the course that depends on it.
3. Apply topological sorting using BFS (Kahn's algorithm) or DFS.
4. If all courses are processed, return true.
5. If a cycle exists, return false because the prerequisites cannot all be satisfied.

Example with a cycle

If Course 0 requires Course 1 and Course 1 requires Course 0, the graph contains the cycle 0 → 1 → 0. Neither course can be completed first, so the answer is false.

3. Difference Between Topological Sort and Course Schedule

Topological Sort	Course Schedule
Finds an ordering of vertices.	Checks whether all courses can be completed.
Used on a DAG.	Uses a prerequisite graph and cycle detection.

Topological Sort	Course Schedule
Returns a valid order.	Usually returns true or false.
Common methods: BFS and DFS.	Common methods: BFS (Kahn’s) and DFS.

4. Applications

• Course prerequisite planning.

• Task scheduling.

• Software build systems.

• Dependency management.

• Project planning.

Conclusion: Topological sorting is possible only for a directed graph with no cycles (DAG). The Course Schedule problem uses topological sorting or cycle detection to determine whether a valid course order exists.